PLURIONE

INFORME TÉCNICO

Ejecución, resultados, pruebas y cierre documental al 17 de octubre de 2026

Plataforma de Observabilidad de Datos con Inteligencia Artificial

Prototipo académico con simulación de telemetría, monitoreo en tiempo real y asistente J.A.R.V.I.S. 2.0.

Desarrollador	Diego Rivera Aguilar
Empresa de referencia	PluriOne S.A. de C.V.
Periodo documentado	9 de septiembre al 17 de octubre de 2026
Fecha de corte	17 de octubre de 2026
Versión	Actualización posterior a la integración del Pull Request #6

Nota de alcance: El proyecto se valida como demostración académica local; los pipelines y las métricas son simulados. No se presenta como implementación productiva en infraestructuras empresariales.

Resumen ejecutivo

Entre el 9 de septiembre y el 17 de octubre de 2026 se documentó la construcción y consolidación de PluriOne, prototipo académico de observabilidad para pipelines de datos. La solución integra FastAPI, PostgreSQL, React/Vite, WebSockets autenticados y J.A.R.V.I.S. 2.0 mediante Groq. Durante la iteración final se comprobó el flujo de recuperación de acceso desde React hasta el inicio de sesión con la contraseña restablecida, además de los rechazos HTTP 401 a accesos no autorizados.

La información monitoreada procede de un simulador interno que genera métricas periódicas. Los diagnósticos locales y las respuestas del modelo constituyen orientaciones técnicas; no demuestran, por sí mismos, la causa raíz de fallos en infraestructura real. La entrega del código se consolidó en la rama main mediante el Pull Request #6, fusionado el 17 de octubre de 2026 en el commit a1aef19.

Resultado principal	Evidencia o alcance
Versión integrada en GitHub	PR #6 fusionado; rama main sincronizada; commit a1aef19.
Acceso protegido	Las rutas /anomalias/ y /chat/ devolvieron HTTP 401 sin JWT; una cuenta inactiva fue rechazada.
Recuperación de acceso	Código SMTP solicitado desde React; restablecimiento y nuevo ingreso comprobados con cuenta de prueba.
Interfaz y telemetría	Dashboard en vivo, feed de alertas y marcas de tiempo; origen simulado.
Calidad de compilación	Compilación de Vite satisfactoria; auditoría de npm sin vulnerabilidades reportadas en la ejecución registrada.

Criterio de evidencia Este informe distingue funciones implementadas, pruebas locales observadas y actividades de mantenimiento o capacitación propuestas. No afirma certificación productiva, pruebas masivas concluyentes ni validación de los criterios del profesor por una herramienta externa.

1. Detección de necesidades

Actividad realizada. Se analizó como problema de referencia la observación tardía de la latencia, los registros con calidad insuficiente y los porcentajes de fallos en flujos de datos. La necesidad planteada fue centralizar la visualización de indicadores, conservar un historial de incidentes y facilitar la interpretación inicial de alertas a través de recomendaciones orientativas. El contexto empresarial de PluriOne S.A. de C.V. sirve como escenario académico; no se documenta una conexión operativa a sistemas corporativos reales.

Las tres familias de métricas definidas fueron latencia en milisegundos, calidad o proporción de datos corruptos y tasa de fallos. Se estableció que el sistema debería asociar cada registro con su pipeline, valor, estado, recomendación, fecha/hora y usuario autorizado.

Entregables. Documento de Requisitos del Producto (docs/PRD.md) y delimitación del Producto Mínimo Viable (docs/MVP.md), acompañados por instrucciones de instalación en los dos archivos README.

2. Propuesta

Actividad realizada. Se definió un prototipo web de monitoreo con backend Python/FastAPI, PostgreSQL, interfaz React y comunicación WebSocket. Para reproducir una carga continua sin depender de infraestructura corporativa, se optó por una fuente interna de telemetría simulada, con envío de eventos a la cuenta correspondiente. La propuesta incorporó un asistente de IA para formular preguntas sobre anomalías y recibir hipótesis de análisis, manteniendo la decisión técnica final en manos del operador.

- Adquisición: simulador asíncrono en el backend; no conectores productivos de AWS, ERP o sistemas externos.
- Almacenamiento: tablas users y anomaly_logs mediante SQLAlchemy y PostgreSQL.
- Presentación: dashboard, historial y reportes PDF en React.
- Análisis asistido: modelo Groq con herramientas SQL previamente autorizadas y filtradas por usuario.

Entregable. Propuesta técnica implementada como prototipo local y documentada en el PRD, MVP, README y el presente informe.

3. Arquitectura

Actividad realizada. Se organizó la aplicación en capas diferenciadas: interfaz web React/Vite, API FastAPI, base de datos PostgreSQL, servicios de correo SMTP, modelo conversacional alojado por Groq y WebSockets para distribución de eventos. Docker Compose se utilizó en las pruebas locales para los servicios API y base de datos; el frontend se ejecutó mediante Vite por separado.

Capa	Implementación	Responsabilidad
Interfaz	React, Vite, React Router, Axios, Recharts	Acceso, visualización de tres gráficos, feed, historial, chat y exportación PDF.
API y seguridad	Python, FastAPI, JWT, hashing	Autenticación, control de cuenta activa, endpoints REST y WebSocket autenticado.
Datos	PostgreSQL, SQLAlchemy	Persistencia relacional y aislamiento por user_id.
Telemetría	Proceso asyncio periódico	Generación de anomalías simuladas; envío a usuarios activos.
Diagnóstico conversacional	Groq, openai/gpt-oss-20b, function calling	Interpretación y consulta controlada de las anomalías del usuario.

Notificaciones de identidad	SMTP	Envío de códigos de verificación y recuperación, no de alertas masivas.

3.1. Flujo de datos y control de acceso

El simulador genera una métrica, la clasifica y la almacena en anomaly_logs. Cada evento tiene user_id. El backend envía un mensaje JSON por WebSocket únicamente a la conexión autenticada del usuario correspondiente, incluyendo la marca de tiempo almacenada. El frontend actualiza las gráficas y el feed sin una recarga manual.

La autenticación se basa en JWT. Para rutas protegidas, el backend verifica la firma, localiza la cuenta y rechaza a usuarios inexistentes o inactivos. La consulta del asistente consultar_anomalias_usuario recibe user_id desde la sesión del backend y no desde un parámetro controlado libremente por el cliente.

3.2. Alcance y límites arquitectónicos

Los pipelines exhibidos en pantalla son ejemplos del simulador. No se afirma que exista un despliegue real de microservicios empresariales, ingesta desde fuentes de producción, remediación automática o infraestructura de alta disponibilidad. Las dependencias externas SMTP y Groq requieren variables de entorno válidas y conectividad.

Entregable. Arquitectura implementada y descrita mediante el árbol del repositorio, los modelos, las rutas de FastAPI, el archivo docker-compose.yml y los diagramas textuales del PRD.

4. Planificación

Actividad realizada. Se trabajó con una adaptación individual de Scrum, organizada por objetivos e iteraciones, sin atribuir reuniones de equipo ni revisiones formales no evidenciadas. La bitácora de desarrollo conserva los periodos originales y extiende el registro al cierre del 17 de octubre.

Sprint	Periodo	Resultado esperado / obtenido
1. Fundamentos y BD	09–22 sep 2026	PRD/MVP, modelos relacionales y entorno PostgreSQL.
2. Backend y seguridad	23 sep–04 oct 2026	API FastAPI, JWT, verificación SMTP y recuperación inicial.
3. Frontend y tiempo real	05–10 oct 2026	Dashboard React, Recharts, WebSockets, PDF y asistente.
4. Contenedores y documentación	11–13 oct 2026	Docker Compose, repositorio Git y documentación inicial.
5. Correcciones y cierre técnico	14–17 oct 2026	J.A.R.V.I.S. 2.0, seguridad por usuario, recuperación desde React, QA y merge PR #6.

Entregable. Bitácora de desarrollo ágil actualizada y trazabilidad de cambios en el historial de Git, con evidencia de la integración final.

5. Desarrollo

5.1. Backend, seguridad e identidad

Se desarrollaron los modelos User y AnomalyLog y las operaciones de registro, verificación de correo, inicio de sesión, solicitud de recuperación, restablecimiento de contraseña y consulta de anomalías. Las contraseñas se almacenan como hashes; el usuario debe activar la cuenta antes de acceder. La corrección final incorporó en get_current_user una comprobación adicional de is_active para impedir el acceso mediante un token aún válido de una cuenta inactiva.

El endpoint /solicitar-recuperacion/ genera y remite un código SMTP al correo registrado. El endpoint /recuperar-password/ exige el código y actualiza el hash de la contraseña; después el código se invalida. En la versión integrada no se implementó expiración temporal ni límite de intentos para esos códigos. Esta limitación se registra explícitamente para no sobredimensionar la seguridad.

Ruta / recurso	Uso constatado
POST /registro/ y POST /verificar/	Registro de cuenta y activación por código SMTP.
POST /login/	Autenticación y emisión de JWT a cuenta activa.
POST /solicitar-recuperacion/	Solicitud de código de recuperación desde React.
POST /recuperar-password/	Cambio de contraseña con código correcto.
GET /anomalias/	Lectura protegida de anomalías del usuario.
POST /chat/	Consulta conversacional protegida con J.A.R.V.I.S.
WebSocket /ws/anomalias	Recepción de anomalías del usuario autenticado.

5.2. Simulador, persistencia y panel

El backend incorpora un proceso asíncrono de generación de métricas simuladas aproximadamente cada cinco segundos. Los registros contienen nombre de pipeline, métrica, valor, nivel de estado, recomendación y marca de tiempo. El dashboard representa latencia con una gráfica de líneas, calidad con barras y fallos con un gráfico de dona; además incluye diagnósticos automáticos locales, feed de alertas e historial.

Durante la revisión final se corrigió la inclusión del campo timestamp en el JSON enviado por WebSocket. Con ello, los eventos recientes pueden mostrar una hora de registro en el feed en lugar de un marcador vacío. En el frontend se ajustó el espacio del chat y de las tablas largas para conservar las gráficas en sus columnas.

5.3. J.A.R.V.I.S. 2.0 y consultas SQL controladas

Se integró el modelo openai/gpt-oss-20b a través de Groq para generar respuestas en español con contexto conversacional reciente. Mediante function calling, el modelo puede solicitar la ejecución de consultar_anomalias_usuario, herramienta que consulta PostgreSQL usando la identidad autenticada en FastAPI. Los filtros disponibles son normal, alerta, critico o todos, y el número de registros devueltos está limitado a un máximo de 10 por consulta.

Las respuestas diferencian datos observados, hipótesis de causa y medidas de diagnóstico propuestas. No se implementaron filtros por rango de fechas o por métricas en la herramienta actual, ni se autorizó la ejecución de comandos destructivos o cambios a infraestructura. La interfaz utiliza react-markdown y remark-gfm para presentar listas, tablas y explicaciones, con desplazamiento contenido y un botón para regresar al último mensaje.

5.4. Versionamiento y revisión de código

Se trabajó en la rama fix/jarvis-groq-usage con commits de revisión y consolidación. La corrección para solicitar códigos desde React quedó registrada en b5f8ff8. El Pull Request #6 se fusionó en main el 17 de octubre de 2026 mediante el commit a1aef19. Posteriormente se verificó git status --short --branch sin modificaciones pendientes y git log -1 --oneline apuntando a ese commit.

Entregable. Código fuente del prototipo en el repositorio, documentación actualizada y funcionalidades verificadas localmente. Se distingue la documentación adicional preparada tras el merge: su publicación se debe confirmar mediante un commit/push específico antes de darla por reflejada en GitHub.

6. Pruebas

Actividad realizada. Se realizaron pruebas locales manuales y verificaciones de compilación, conectividad, autenticación y funcionamiento del prototipo. Los resultados siguientes proceden de comandos y observaciones de la iteración, no de una suite completa de pruebas automatizadas.

Prueba o comprobación	Resultado observado	Límite de interpretación
Compilación Python	python -m py_compile completó sin errores.	Valida sintaxis, no funcionamiento total.
Compilación React	npm --prefix .\frontend run build: 1453 módulos; built correctamente.	Advertencia por chunks mayores de 500 kB, no bloqueante.
Auditoría npm	npm audit --audit-level=high: found 0 vulnerabilities.	Resultado del momento de ejecución, sujeto a avisos futuros.
Docker Compose	API y PostgreSQL en estado Up en docker compose ps.	Disponibilidad local, no SLA productivo.
API sin token	GET /anomalias/: HTTP 401.	Ruta protegida correctamente en esta prueba.

Chat sin token	POST /chat/: HTTP 401.	Se rechaza la petición antes de consultar Groq.
JWT de cuenta inactiva	Solicitud protegida con token de cuenta inactiva: HTTP 401.	Prueba puntual del control is_active.
Recuperación SMTP	Código recibido; cambio de contraseña e inicio de sesión realizados.	Prueba manual con cuenta de prueba.
Telemetría / interfaz	Gráficas, feed, horas, historial y chat observados en navegador.	Telemetría sintética, no datos corporativos.
Respaldo BD	pg_dump personalizado y pg_restore --list ejecutados; archivo de 245275 bytes.	Se validó la estructura del respaldo; no consta una restauración completa de ensayo.

6.1. Incidencias observadas y resolución

- Una consulta inicial de recuperación devolvió «Código incorrecto o expirado»; al solicitar un código nuevo y utilizar el más reciente se comprobó el cambio y el posterior inicio de sesión.
- Durante pruebas de desarrollo, el navegador mostró «ERR_CONNECTION_REFUSED» cuando Vite estaba detenido; se resolvió iniciando nuevamente npm run dev.
- Se eliminaron espacios sobrantes detectados por git diff --check antes de consolidar cambios.
- Groq presentó límites de cuota o respuestas temporales de servicio en ciertas iteraciones; su disponibilidad sigue siendo una dependencia externa.

6.2. Pruebas no acreditadas

No se cuenta con evidencia para declarar superadas pruebas formales de penetración, carga masiva instrumentada, tolerancia a fallos, auditoría de seguridad externa, pruebas automatizadas de cobertura amplia, conectividad a pipelines reales o recuperación comprobada desde un respaldo restaurado. Estas comprobaciones corresponden a una eventual ampliación del prototipo.

Entregable. Matriz de resultados técnicos reproducibles y constancias de compilación, comportamiento HTTP, operación de interfaz y versionamiento.

7. Documentación

Actividad realizada. Se estructuraron instrucciones técnicas para configuración del entorno, variables de entorno, servicios, rutas de API, mecanismo JWT, comunicación WebSocket, flujo de recuperación, interpretación de alertas y uso de J.A.R.V.I.S. Se editaron el README de raíz, el README del frontend, el PRD y el MVP para conservar el alcance académico y reconocer el origen simulado de los datos.

Documento	Contenido / ubicación
README.md	Visión general, stack tecnológico, variables de entorno, ejecución y limitaciones.

frontend/README.md	Uso de React, rutas, dependencias y comunicación con FastAPI.
docs/PRD.md	Problema, alcance, historias de usuario, arquitectura y criterios de aceptación.
docs/MVP.md	Funcionalidades implementadas, evidencias de pruebas y exclusiones.
Bitácora Scrum actualizada	Secuencia de sprints, resultados y retrospectivas hasta el 17/10/2026.
Informe Técnico actualizado	Descripción de diez fases, pruebas y estado real del prototipo.

Entregable. Manual técnico y de usuario distribuido entre los documentos del repositorio y el presente informe. Los archivos nuevos o revisados no quedan publicados en GitHub hasta realizar y verificar su respectiva incorporación al repositorio.

8. Mantenimiento

Actividad realizada. Se atendieron defectos encontrados durante el desarrollo, incluyendo presentación del chat, generación de marcas de tiempo WebSocket, controles de JWT y recuperación desde la interfaz. Se ejecutaron comprobaciones de dependencias y se generó un respaldo de PostgreSQL antes de explorar una ampliación opcional del esquema de códigos de seguridad.

8.1. Plan recomendado de soporte

- Nivel 1: revisar disponibilidad de los servicios con docker compose ps y logs operativos de FastAPI; supervisar el crecimiento de los datos simulados.
- Nivel 2: analizar errores de autenticación, conexión PostgreSQL, WebSocket y SMTP; evaluar alertas de dependencias con npm audit y actualizaciones controladas.
- Evolutivo: incorporar pruebas automáticas, medidas de expiración y limitación de intentos en códigos SMTP, y mejorar métricas de confiabilidad y despliegue solo si se autoriza una nueva fase.

El esquema exploratorio de seguridad para códigos (columnas de vencimiento, intentos y propósito) se aplicó en una base local durante una prueba, pero su código y su migración fueron retirados de la versión Git principal. Por tanto, no se considera una capacidad del sistema publicado.

Entregable. Lineamientos de mantenimiento y evidencia del respaldo local. No consta un servicio empresarial formal de soporte en producción.

9. Capacitación

Actividad realizada. Se organizó una guía de uso y demostración de las pantallas principales para una posible presentación académica: registro/verificación, inicio de sesión, recuperación, lectura de

métricas, historial, exportación y preguntas a J.A.R.V.I.S. Esta preparación no equivale a una capacitación institucional formal ni a la emisión de un acta firmada.

Perfil	Contenido de demostración
Operador / usuario	Ingreso, interpretación de estados Normal/Alerta/Crítico y consulta del feed.
Analista de datos	Lectura de latencia, calidad y fallos; preguntas basadas en registros del usuario.
Administrador técnico	Variables de entorno sin secretos expuestos, estado de Docker, JWT, SMTP y PostgreSQL.
Evaluador académico	Delimitación de datos simulados, demostración del MVP y revisión de resultados documentados.

Entregable. Guía operativa para demostración del prototipo; queda pendiente un acta de capacitación si fuera requisito formal del destinatario.

10. Cierre

Actividad realizada. Se completó la consolidación técnica de la iteración de PluriOne y se verificó el estado final de la rama main después de fusionar el Pull Request #6. El proyecto demuestra un recorrido funcional desde la generación automática de telemetría simulada hasta su almacenamiento, visualización, consulta asistida por IA y recuperación de acceso mediante correo.

El avance comprobado es significativo para el MVP académico. No se afirma que los requerimientos hayan sido certificados en su totalidad por el profesor ni que el prototipo cuente con validación de rendimiento o seguridad para un entorno público. La entrega documental deberá publicarse y revisarse junto con el código para preservar la coherencia de las evidencias.

10.1. Lecciones aprendidas

- Separar la telemetría sintética de las fuentes corporativas reales evita exagerar el alcance del proyecto.
- Es necesario comprobar la autenticación en todas las rutas, el chat, los WebSockets y los estados de cuenta, no únicamente en el inicio de sesión.
- La recuperación de contraseña requiere dos operaciones completas en el frontend: solicitar el código y utilizarlo para restablecer el acceso.
- Las respuestas del modelo deben basarse en herramientas autorizadas y distinguir hipótesis técnicas de causas verificadas.
- El versionamiento con PR y el respaldo de datos facilitaron revertir una mejora opcional sin afectar la versión estable.

10.2. Estado de entrega y pendientes delimitados

Elemento	Situación al 17/10/2026
Código principal	Integrado en main; commit a1aef19; sincronización local confirmada.
Documentación funcional	README, README frontend, MVP y PRD existentes; última edición local debe publicarse y verificarse.
Informe y bitácora	Actualizados con fecha de corte 17/10/2026; listos para revisión y publicación posterior.
Alcance real	Prototipo académico con datos simulados y servicios externos SMTP/Groq.
Trabajo futuro	Auditoría integral, pruebas automáticas, recuperación con expiración y límites, conectores reales si se solicita.

Conclusión. La versión presentada evidencia la integración funcional de FastAPI, React, PostgreSQL, WebSockets y Groq en un escenario de observabilidad simulado. Este informe documenta el trabajo realizado y el estado comprobable hasta el 17 de octubre de 2026, manteniendo separados los entregables implementados y los objetivos aún no acreditados.

Fuentes de trazabilidad y anexos consultados

- Bitácora de Desarrollo Ágil (Scrum) - Proyecto PluriOne.pdf, versión original con periodo 09/09–13/10/2026, proporcionada por el desarrollador.
- Informe Técnico Plataforma de Observabilidad.pdf, versión original organizada en diez apartados, proporcionada por el desarrollador.
- README.md, frontend/README.md, docs/MVP.md y docs/PRD.md de PluriOne; documentación de la iteración final.
- Repositorio del proyecto: https://github.com/Diego-riveera02/G19X-TESE-DRA-122-ACADEMIC. Pull Request #6, fusión a1aef19, 17/10/2026.
- Registros de terminal y capturas de pruebas locales compartidas durante la iteración del proyecto, hasta el 17/10/2026.